TimeTrack

Complete guide: the application, how it is built and tested, how changes reach production, how data gets into the database, and every third-party service involved

Live demo: https://timetrack-demo-kanak.vercel.app
Source: github.com/kanakverma89/timetrack (private repository)
Written 25 October 2026, from the repository as it stood at the end of Phase 7 and the free-tier deployment. Free-tier limits quoted here were read from the providers' own pages on that date. They change; check the current terms before relying on them.

Contents

1. What TimeTrack is
2. What it does, role by role
3. How it is built (architecture)
4. All the scenarios
5. GitHub and its workflows
6. The process of making a change
7. How data gets into the database
8. How to deploy: the full process
9. Third-party services and tools in detail
   Neon · Vercel · Render · GitHub · Docker images · libraries · test and security tools
10. Running it afterwards: operations, costs, limits, risks
11. Appendix: commands, file map, glossary

1. What TimeTrack is

TimeTrack is a **weekly timesheet application**. People record the hours they worked against projects (and the tasks inside them), submit the week, and a manager approves or rejects it. Administrators manage the people and reference data and can read an audit log. Managers and administrators can run reports.

It was built as an independent, neutral-branded learning project from a written specification (`spec.md`), in seven phases, with real automated tests at every layer. It is not a copy of any company's product.

The life of a week

```
DRAFT ──submit──▶ SUBMITTED ──approve──▶ APPROVED   (final, read-only)
   ▲                  │
   └── (edit, fix) ◀── reject  (REJECTED: editable again, then resubmit)
```

Three roles

Role	Who	Can
Employee	Anyone who logs time	Log, edit, delete and submit their own hours; change their own password.
Manager	Someone with direct reports	Everything an employee can, plus review, approve or reject the weeks of the people who **report to them**, and run reports for those people.
Administrator	Whoever runs the system	Everything a manager can for **everyone**, plus create and manage users, teams, projects, tasks, earning codes, hour policy, and read the audit log.

> **Two rules that surprise people.** (1) Team membership gives *no* approval authority: only a person's assigned manager (or an administrator) can approve their week. (2) Nobody can approve their *own* week, not even an administrator.

The size of the project

Measure	Value
Phases delivered	0 to 7 (foundation, dashboard, day-mode entry, week-mode entry, submission, approvals, administration/reporting/hardening), plus a free-tier deployment
Automated tests	540 backend (pytest, real PostgreSQL), 254 component tests (Vitest), 369 end-to-end tests (Playwright on Chromium, Firefox and WebKit; 362 pass, 7 are deliberately skipped)
Database migrations	5 (Alembic revisions 0001 to 0005)

Documentation	About 1,800 lines in `docs/`: architecture, 72 numbered design decisions, API contracts, data model, testing strategy, security review, operations guide, performance report, user guide, free-tier deployment guide

2. What it does, role by role

Employee

- **Timesheet dashboard.** Shows one week at a time with a card per day, a weekly summary (total, capped, non-capped, draft entries) and the entry list. A week navigator moves between weeks; the address (`?week=YYYY-MM-DD`) can be bookmarked.
- **Day mode** (add one entry): date, hours (up to two decimals, above 0, at most the policy maximum, 24 by default), project (searchable by code or name), task (required when the project has active tasks), earning code and a description.
- **Week mode** (add several days at once): one project/task/code and an hours value per day, saved **all or nothing**. Errors point at the day that failed; "already logged" figures stop accidental double entry. It only adds; it never replaces existing entries.
- **Edit and delete** entries while the week is a draft or rejected. An entry carries a version number, so if it changed elsewhere you get a "changed elsewhere" message instead of silently overwriting.
- **Submit.** A dialog shows totals and takes an optional comment. The server re-checks every entry against the current policy (hours per entry, per day, per week) and either submits the whole week or explains exactly what to fix. After submission the week is read-only.
- **Rejections.** A banner shows the reason, who rejected it and when; the week becomes editable; **Resubmit week** sends it again.
- **Change password** (12+ characters, not a common password, not containing the name or email). Other devices are signed out.

Manager

- **Approvals** list with a badge counting weeks waiting; filter by status (waiting, approved, rejected) and by week; oldest submission first; paged.
- **Review page**: employee, every entry, their comment, and the history of previous decisions.
- **Approve** (optional comment) or **Reject** (a reason of at least 5 characters is required and shown to the employee). Two reviewers acting at once produce exactly one decision.
- **Reports**: hours by person, project, earning code or week, for a date range of up to 366 days; Approved weeks by default (Submitted and Rejected can be ticked in); drafts are never counted; totals cover all rows; **CSV export** of the same query.

Administrator

- **Users.** Create a person (email, name, role, optional manager). The server generates a **temporary password shown exactly once**; the person must replace it at first sign-in. Edit name, role and manager; deactivate or reactivate (sessions end immediately); reset a password.
- **Safety rails.** You cannot change your own role, deactivate yourself or reset your own password; the last active administrator cannot be removed (race-safe); a manager with active reports cannot be

deactivated until the reports are reassigned.

- **Teams** (organisational grouping only), **Projects and Tasks**, **Earning codes** (capped or non-capped; the classification is locked once any entry uses the code).
- **Hour policy**: maximum hours per entry, per day, per week and description length. It overrides the server's configured defaults and applies at once; *Restore defaults* removes the override.
- **Audit log**: every administrative change, newest first, with who did it and before/after values. It stores people by id and shows their *current* names, never passwords, so an anonymised person reads "Former employee".

Cross-cutting behaviour

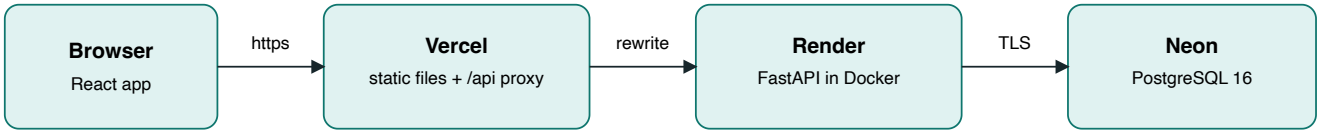
- **Accessibility.** Real tables that turn into labelled cards on phones, a focus-trapping modal that returns focus, errors that receive focus, automated axe checks on every page and dialog. (No screen reader or real phone was used in testing.)
- **Safety against mistakes.** Double-clicks send one request; submissions and decisions accept an `Idempotency-Key` so a retry is harmless; a week is locked in the database while it changes.
- **Notifications.** A submission or decision writes a row to an outbox table; a background step hands it to a delivery adapter (currently a log line, no email).

3. How it is built (architecture)

The stack

Layer	Technology	Why it is there
Frontend	React 19, TypeScript, Vite, TanStack Query, React Router	The user interface. TanStack Query handles server data (caching, retries). Hand-written accessible components, no UI kit.
Backend	Python 3.12, FastAPI, Pydantic, uvicorn	The HTTP API, validation and business rules.
Data access	SQLAlchemy 2 (synchronous), psycopg 3, Alembic	Queries and transactions; Alembic applies schema changes (migrations).
Database	PostgreSQL 16	Row locks, triggers (append-only audit), CHECK constraints, JSON columns.
Passwords	Argon2id (argon2-cffi)	Slow, memory-hard hashing so stolen hashes are expensive to crack.
Testing	pytest, Vitest, Playwright, axe-core	Backend, component, browser and accessibility tests.

The live deployment: one origin for the browser



The browser only ever talks to the Vercel address. Vercel forwards /api/* to Render, so the session cookie (SameSite=Lax) is always sent and the API's CORS/CSRF origin check sees one origin. Render connects to Neon over TLS.

Inside the backend

Code flows downwards only: **api** (routes and request/response shapes) → **services** (business rules and transactions) → **repositories** (queries) → **models** (tables). Reference-data routes call repositories directly, because a pass-through service would add nothing.

- **Authentication.** Email and password; passwords hashed with Argon2id. A successful sign-in creates a server-side session; the browser holds a random token in an `HttpOnly`, `SameSite=Lax` cookie (`Secure` in production). The database stores only an HMAC digest of the token, so a database leak alone yields no usable sessions.

- **CSRF protection.** Every state-changing request must carry `X-Requested-With: timetrack` and, if it has an `Origin` header, that origin must be in the `CORS_ORIGINS` allow-list. A cross-site page cannot add a custom header.
- **Authorization.** Role guards on whole routers (admin, reports, approvals) and one pure function (`review_policy`) that decides who may review whom. A test lists *every* route with its allowed roles and fails if a route is missing.
- **Error envelope.** Every error has one shape: `{"error": {"code","message","field_errors","request_id"}}`. Unknown or foreign resources return 404 (not 403) so ids cannot be probed.
- **Concurrency.** A week row is locked `SELECT … FOR UPDATE` while it changes; entries carry an optimistic version; idempotency keys are stored in their own table.
- **Audit.** Three tables (`approval_events`, `time_entry_events`, `config_audit_events`) are protected by a database trigger that refuses UPDATE and DELETE, so even a buggy deployment cannot rewrite history.
- **Configuration.** All settings come from environment variables. In production mode the app *refuses to start* with a short or placeholder secret, a placeholder database password or an origin that is not `https://`, and says which.

The main tables

Table	Holds
users, auth_sessions	Accounts (with `manager_id`, `must_change_password`) and login sessions (token digests only).
teams, team_memberships	Organisational groups.
projects, project_tasks, earning_codes	What time can be booked against; classification capped / non-capped.
timesheet_weeks, time_entries	One row per employee per week; entries with hours as `NUMERIC(5,2)`, never floating point.
approval_events, time_entry_events, config_audit_events	Append-only history.
notification_outbox, idempotency_keys	Reliable notification hand-off; safe retries.
policy_settings	One row: the administrator's hour policy.
alembic_version	Which migration the database is at (currently 0005).

Environments

Where	Database	Used for
Your laptop (development)	`timetrack` in a local Docker PostgreSQL	Manual use with seeded synthetic data (`scripts/dev-up.sh`).

Tests	`timetrack_test`, recreated every run	pytest.
End-to-end	`timetrack_e2e`, recreated and seeded every run	Playwright. Scripts refuse to drop any database whose name does not end in `_test` or `_e2e`.
Live demo	Neon project `timetrack-demo`, database `neondb`	The public deployment.

4. All the scenarios

The demo accounts are synthetic: `employee@example.test` (Alex), `employee2@example.test` (Sam), `employee3@example.test` (Taylor, who reports to a different manager), `manager@example.test` (Morgan, manages Alex and Sam), `manager2@example.test` (Riley, manages Taylor) and an administrator. All share the password set when the data was seeded; the published employee and manager logins are in the README.

Tip: use two browser profiles (or a normal and a private window) to be signed in as an employee and their manager at once.

A. The happy path (Alex, Morgan)

1. Alex opens the current week, adds entries in Day mode and Week mode, then clicks **Submit week**.
2. Morgan sees the week under **Approvals** (badge count), opens **Review** and clicks **Approve**.
3. Alex refreshes: the banner says approved, and Add/Edit/Delete are gone.
4. Morgan opens **Reports** for that week: Alex's hours appear (Approved is the default filter).

B. Rejection and resubmission

1. Alex submits. Morgan chooses **Reject** with a reason such as "Please recheck Tuesday's hours."
2. Alex sees the rejection banner and reason, fixes Tuesday and clicks **Resubmit week**.
3. Morgan approves. The review page history shows submitted → rejected → submitted → approved.

C. Onboarding a new hire (administrator)

1. Admin → Users → **Add user**: email, name, role Employee, manager Morgan. A dialog shows the temporary password once; copy it and hand it over privately.
2. The new hire signs in with it and is taken to **Choose a new password**; nothing else in the app works until it is changed (the API enforces this too, with a 403 on every other route).
3. Their submitted weeks appear for Morgan and not for Riley.

D. A leaver

The administrator unticks **Active** on the person. Their open session ends at once, they cannot sign in (the message is the same as for a wrong password), their history stays. Reactivating restores access with the same password. If the identity must also be removed later, `scripts/maintain.py anonymise-user` replaces name and email with "Former employee" while keeping hours and decisions.

E. A forgotten password

Admin → Users → **Reset password**. A new one-time password is shown; the person is signed out everywhere and must change it at next sign-in. An administrator cannot reset their own password this way

(they use *Change password*).

F. Changing the rules

1. Admin → Policy: set the per-entry maximum to 8. Alex tries to save 9 hours → refused with the limit named.
2. A previously saved 10-hour entry now blocks submission until fixed (submission re-checks every entry).
3. **Restore defaults** removes the override. Both changes are in the Audit log with before and after values.

G. A manager's reach

Taylor (reports to Riley) submits. Morgan does **not** see it, Riley does, the administrator does. Morgan's reports include Alex and Sam only. Asking for Taylor's data by id returns an empty result or 404, never the data.

H. Retiring a project

Add a project with a task; Alex can pick it and must choose the task. Deactivate the project: it disappears from the entry form but Alex's old entries still show it.

I. Edge cases the tests cover

Situation	What happens
Two browser windows edit the same entry	The second save gets a "changed elsewhere" message (HTTP 409).
Double-click on Save, Submit or Create user	One request, one record.
Two managers decide the same week at once	One wins; the other is told it was already decided.
Week mode with one invalid day	Nothing is saved; the error points at that day.
Employee opens an admin address	"You do not have access" in the page and 403 from the API.
Five wrong passwords for one account	429 "Too many failed attempts" for 15 minutes.
Request without the CSRF header, or from a foreign origin	403.
Hours of 25, 7.555 or 0	Field error naming the rule.
Last administrator removed (two at once)	Refused (row lock; reproduced by a two-thread test).
Spreadsheet formula in a project name (`=1+1`)	CSV cell is exported with a leading quote so Excel does not run it.

5. GitHub and its workflows

The code lives in a **private GitHub repository**, `kanakverma89/timetrack`. GitHub does three jobs here: it stores the history, it runs **GitHub Actions** (automated checks), and it is the source that Vercel and Render read from when they deploy.

The three workflows

Workflows are YAML files in `.github/workflows/`. Each runs on a fresh Ubuntu 24.04 virtual machine that GitHub provides.

1. `ci.yml` — continuous integration

Trigger	Every push to any branch, and every pull request. A newer push to the same branch cancels an older, still-running run.
Services	A PostgreSQL 16 container (the backend and end-to-end jobs need a real database).

Job	Runs on	Steps
backend	Python 3.12 and 3.14 (a matrix: two parallel runs)	`pip install -e ".[dev]"`, `ruff check`, `ruff format --check`, `mypy` (strict), `alembic upgrade head` then `alembic check` (migrations apply from an empty database and match the models), `pytest` (540 tests).
frontend	Node 22, 24 and 26	`npm ci`, ESLint, Prettier check, TypeScript check, Vitest (254 tests), production build.
e2e	Python 3.12 + Node 22, after the two jobs above pass	Installs Chromium, Firefox and WebKit, then runs the full Playwright suite (369 tests) against a real API and database. Takes roughly 16 to 21 minutes on GitHub's runner; the job timeout is 45 minutes. Uploads the Playwright report if anything fails.

A green run means: code style and types are clean, every migration applies, and all backend, component and browser tests pass on all supported Python and Node versions.

2. `audit.yml` — dependency audit

Trigger	Every Monday at 05:17 UTC; manually; and on pushes that change `backend/pyproject.toml`, `frontend/package.json`, `frontend/package-lock.json` or the workflow itself.
Jobs	**python**: installs the backend and runs `pip-audit --skip-editable`. **node**: `npm ci` then `npm audit --audit-level=low` (production and development dependencies).
Why separate	A newly published vulnerability should become visible without turning every unrelated push red.

3. `deploy-verify.yml` — live smoke test

Trigger	**Manual only** (`workflow_dispatch`): Actions tab → deploy-verify → Run workflow, or `gh workflow run deploy-verify` .
Needs	A repository **variable** `DEMO_URL` (the live frontend address) and a repository **secret** `DEMO_EMPLOYEE_PASSWORD` (the demo password). Settings → Secrets and variables → Actions. Secrets are encrypted and never printed in logs.
What it does	Installs Chromium and runs six smoke tests against the *deployed* site (`frontend/tests/smoke/deployed.spec.ts`): the API answers through the frontend address; the sign-in page loads; docs and registration do not exist, security headers are present and the shipped JavaScript contains no connection strings; anonymous callers are refused and an employee is refused on admin, approvals and reports; signing in works and the session cookie is `HttpOnly` , `Secure` , `SameSite=Lax` ; an entry is saved, read back and deleted.
Result recorded	6 of 6 passed against the live demo on 25 October 2026.

> **Deployments are not GitHub Actions.** Vercel and Render deploy through their own GitHub integrations: when `main` changes, each notices, builds and releases by itself. GitHub Actions only *checks* things; it holds no deployment credentials, which is why no cloud tokens are stored in the repository.

Branches and history

- One branch per piece of work (for example `feat/phase-7-administration-and-hardening` , `feat/free-tier-deployment`). `main` is the released state.
- A branch is merged into `main` with `git merge --no-ff` (a merge commit that keeps the branch visible in history), only when explicitly asked.
- Commits use GitHub's no-reply email address (GitHub refuses pushes that expose a personal address) and end with a `Co-Authored-By` line.

6. The process of making a change

Every change in this project followed the same loop. The deployment itself produced two real examples, described below.



8 Verify the live site (deploy-verify) · 9 If wrong: roll back in Vercel/Render, fix, repeat

The steps in detail

1. **Branch.** `git checkout -b feat/short-name` from an up-to-date `main`.
2. **Change.** Edit code *and* write the test that proves it. Rules that matter get a test that fails if the rule is removed (a deliberate "mutation check": break the code, confirm a test goes red, restore it).
3. **Check locally, the same commands CI runs:**

```
cd backend  && ruff check . && ruff format --check . && mypy app scripts && pytest
cd frontend && npm run lint && npm run format:check && npm run typecheck && npm test &&
npm run build
cd frontend && npx playwright test        # three browsers, about 8 minutes
```

4. **Commit and push.** `git add` only the intended files, commit with a message that explains *why*, `git push -u origin <branch>`.
5. **CI.** GitHub Actions runs `ci.yml` on the branch. Read failures, fix, push again.
6. **Merge.** `git checkout main && git merge --no-ff <branch> && git push origin main`, only when the owner says to.
7. **Deploy.** Pushing `main` makes Vercel rebuild the frontend and Render rebuild and restart the API, automatically.
8. **Verify.** Run `deploy-verify` and click through the real flow.
9. **Roll back if needed.** Vercel: Deployments → promote a previous one. Render: Deploys → roll back. Migrations only move forward, so roll the code back only if the old code still works with the current schema; otherwise restore data (see section 10).

Real example 1: pinning the region (a config-only change)

The Render form for the Blueprint did not let us choose a region, so it would have defaulted to Oregon while the Neon database is in Ohio. Render cannot change a service's region after creation, so the fix had to happen first:

1. Add `region: ohio` to `render.yaml`, validate the file parses, note it in the deployment guide.
2. Commit and push to `main` (commit `a49adfd`).

3. Reload the Render form so it re-read the file from GitHub, re-enter the two secret values, deploy. The service list showed Region: Ohio.

Real example 2: the failed first deploy (a code change driven by a log)

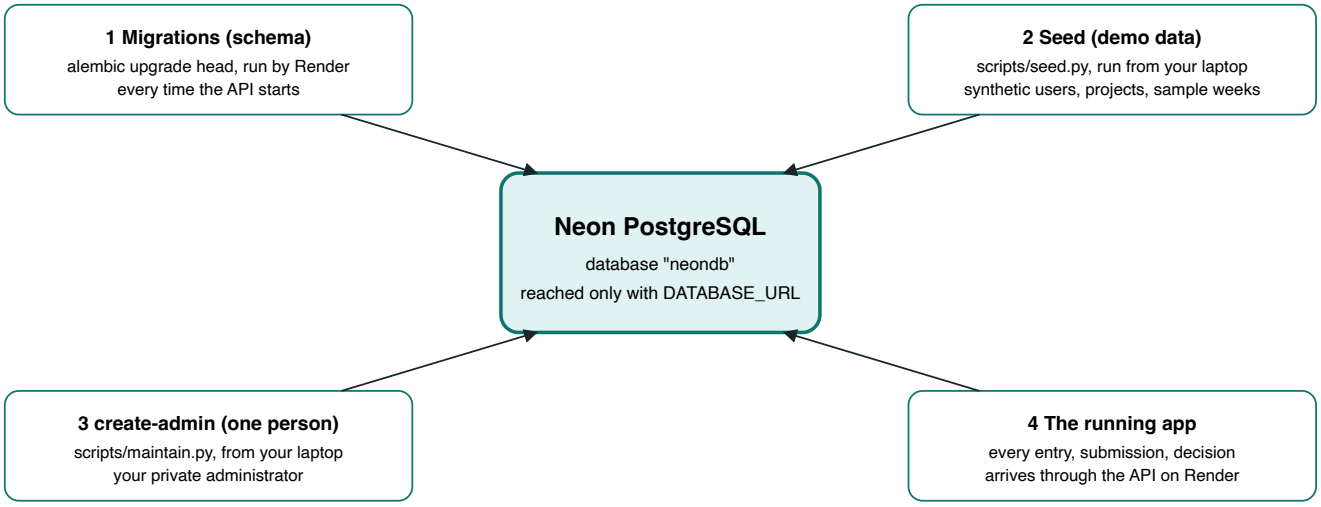
1. **Symptom.** The Render deploy failed in under a minute. The log: `sh: 1: alembic upgrade head && exec uvicorn …: not found`, exit status 127.
2. **Diagnosis.** The Docker image had built fine. Render does not run `dockerCommand` through a shell, so the whole string was treated as the name of one program. My earlier local test had wrapped it in `sh -c` myself, so it could not reproduce this.
3. **Fix.** Put the two commands in `backend/scripts/render-start.sh` and set `dockerCommand: sh scripts/render-start.sh` (two plain words, nothing to mis-parse).
4. **Test before pushing.** Build the real image, run that exact command in production mode against a throwaway database: migrations to 0005, listens on the supplied port, health ok, docs 404, a restart changes nothing. Delete the throwaway database, container and image.
5. **Ship.** Commit `a6f4213` pushed to `main`; Render redeployed; status **Live**.
6. **Lesson recorded** in the deployment guide's troubleshooting table.

Where decisions are written down

`docs/decisions.md` holds 72 numbered decisions (D-1 to D-72), each with its reason. `docs/implementation-report.md` records, for every phase, what was verified, which defects testing found, and what was *not* verified.

7. How data gets into the database

There are four different ways data reaches the production database. Knowing which is which explains almost every step of the setup.



1. The schema: migrations

The tables are not created by hand. They are described by five **Alembic migration** files in `backend/alembic/versions/` (0001 initial schema → 0005 administration, policy and password flag). Each file says how to move the database up one version (and back). The database remembers its version in the `alembic_version` table; `alembic upgrade head` applies whatever it has not seen yet and does nothing when it is current, so it is safe to run repeatedly.

On the live system this happens automatically: Render's start command (`backend/scripts/render-start.sh`) runs `alembic upgrade head` and then starts the server. The first successful deploy therefore created every table in Neon; later restarts change nothing. (The start script printed "Running upgrade → 0001 … → 0005" in my local test of it; for the live system the evidence is that `/api/v1/health/ready` reports the database as ok and the seed later ran successfully, which needs the tables to exist.) That is also why Render's free plan, which has no separate pre-deploy command, still migrates safely. A test in CI (`alembic check`) fails if the Python models and the migrations ever disagree.

2. The demo data: the seed script, from your laptop to Neon

`backend/scripts/seed.py` inserts **synthetic** data: six accounts (employee, employee2, employee3, manager, manager2, admin), two teams, four projects with tasks, five earning codes (one retired), and with `--sample-entries` a few sample weeks in draft, submitted and rejected states.

It runs on **your laptop**, but writes to **Neon**, because the script connects to whatever database `DATABASE_URL` names. The steps:

```
cd backend
# the Neon "direct" connection string, scheme postgresql+psycopg://
read -rs DATABASE_URL ; export DATABASE_URL
# the seed refuses to run when APP_ENV=production
export APP_ENV=development
# the shared demo password you choose (12+ characters)
read -rs SEED_PASSWORD ; export SEED_PASSWORD
.venv/bin/python scripts/seed.py --sample-entries     # prints "Seed complete."
unset DATABASE_URL SEED_PASSWORD APP_ENV
```

Run these one line at a time (see the warning below). Comment lines are for the reader; do not paste them into zsh.

- **Why the laptop and not the server?** The seed deliberately refuses production mode, so it cannot be triggered from the public site. Running it by hand keeps a bulk data load under your control.
- **Why `read -rs`?** It prompts for a value without echoing it or saving it in shell history. The variables exist only in that terminal session, and `unset` removes them.
- **What the password becomes.** The seed hashes `SEED_PASSWORD` with Argon2id (parameters visible in the hash prefix `$argon2id$v=19$m=65536,t=3,p=4$`) and stores *only the hash*. The plain text is never stored or logged.
- **Safe to repeat.** The seed *merges* rows by fixed ids: running it again updates the same six accounts (including resetting their password hashes to the current `SEED_PASSWORD`) instead of duplicating them. That property is what let us repair a mistake (below).
- **Your connection string** is the key to the database. It is typed into a terminal and into Render's dashboard and nowhere else; it is not in the repository, in GitHub, or in Vercel.

> **What went wrong once, and the fix.** Each `read -rs` prompt waits for its *own* value. The first time, the connection string was pasted at every prompt, so the demo accounts and the private administrator all got the connection string as their password (stored only as hashes, so nothing was exposed, but publishing it as "the demo password" would have handed out the database credential). Because the seed merges, the repair was: re-run it with a proper made-up demo password; sign in as the administrator using the old value and use the forced password change to set a real one. Separately, pasting a whole multi-line block into zsh fails because zsh does not treat `#` as a comment interactively and `read` swallows the following pasted lines: run such commands one line at a time.

3. The private administrator

The seed's administrator shares the demo password, so a separate, private one is created with `scripts/maintain.py create-admin EMAIL "Name"` (password from `ADMIN_PASSWORD`). The account is marked *must change password*, so the first web sign-in forces a new password. Afterwards the seeded `admin@example.test` is deactivated in the Admin screen so the public demo password never opens the Admin area.

4. Everything users do: through the API

Once the app is running, data enters only through the API on Render:

1. The browser sends a request to the Vercel address, e.g. `POST /api/v1/time-entries`. The request carries the session cookie and the CSRF header.
2. Vercel's rewrite forwards it to Render (`https://timetrack-demo-api.onrender.com/api/v1/…`).
3. FastAPI validates the body (Pydantic), checks the session and the user's role, and runs the business rules.
4. The service opens a database transaction through SQLAlchemy and the psycopg driver, over TLS to Neon: it locks the week row, checks it is editable, inserts the entry *and* an audit event, and commits both together or neither.
5. The response goes back along the same path.

Hours are stored as exact decimals (`NUMERIC(5,2)`) and travel as strings in JSON, so no floating-point rounding can alter them.

Reading and backing up the data

- **See what is in Neon:** the Neon console's SQL Editor and Tables views, or `psql` with the connection string.
- **Back it up:** `pg_dump -Fc` against the direct connection string. Neon's free plan keeps a 6-hour restore window and one manual snapshot but no scheduled backups, so your own dump is the real backup. `scripts/backup-restore-drill.sh` proves a dump restores correctly (it was run against the development database).
- **Reset the demo:** recreate the Neon database (or branch), redeploy the API so it migrates, then run the seed and create-admin again.

8. How to deploy: the full process

Two deployment paths exist. The **free demo path** (Vercel + Render + Neon) is what is live. The **self-hosted path** (Docker Compose on your own server) is documented in `docs/operations.md`.

8.1 The free demo path, step by step

Step 0: what is already in the repository

File	Purpose
`frontend/vercel.json`	Tells Vercel: build with `npm run build`, serve `dist`; forward `/api/*` to the Render API; fall back to `index.html` for page routes; send security headers (CSP, nosniff, frame denial, referrer and permissions policies).
`render.yaml`	A Render Blueprint: one Docker web service `timetrack-demo-api`, free plan, Ohio, health check `/api/v1/health/ready`, auto-deploy on commit, environment variables (secrets are declared `sync: false` so values are never in the file).
`backend/Dockerfile`	Builds the API image (Python 3.12, non-root user).
`backend/scripts/render-start.sh`	Migrate, then start uvicorn on Render's `$PORT`.
`.github/workflows/deploy-verify.yml`	Smoke tests against the live site.

Step 1: Neon (the database)

1. Sign up at neon.com (GitHub login works). Create a project `timetrack-demo` in **AWS US East 2 (Ohio)**; keep only the Postgres service enabled (not Object storage, Functions, AI gateway, Realtime or Better Auth).
2. Skip the "which coding agent" onboarding question ("I'm not using an agent").
3. **Connect** in the top navigation → turn **Connection pooling off** (the host must not contain `-pooler`) → copy the string.
4. Edit it for this app: `postgresql://` becomes `postgresql+psycopg://` (the psycopg 3 driver), keep `?sslmode=require`, and drop `&channel_binding=require`. Keep it private.

Step 2: Vercel (the frontend)

1. Sign up at vercel.com with GitHub on the **Hobby** (free) plan; give it access to only the `timetrack` repository. Team name is just a label.
2. Add New → Project → import the repository. Vercel notices a frontend and a Python backend and offers a "multi-service project": **decline it** and choose **Import single project** for the `frontend` folder (the API is not hosted on Vercel).

3. Set Project Name (this becomes the address, `https://<name>.vercel.app`), Root Directory **frontend**, preset Vite (it reads `vercel.json`). No environment variables. Deploy (about 20 seconds).
4. Note the **production** address. The other addresses Vercel lists (per-deployment and per-branch URLs) are locked behind a Vercel login by default and have different origins; only the production address is the public one, and it opened without a login.

Step 3: Render (the API)

1. Sign up at render.com with GitHub; stay on the free plan. On the first screen skip the "Key Value instance" onboarding wizard.
2. New → **Blueprint** (not "Web Service" or "Postgres"). Connect GitHub, authorise only the `timetrack` repository, choose branch `main`.
3. Render reads `render.yaml` and lists `timetrack-demo-api`. Enter the two secret values: `DATABASE_URL` (the edited Neon string) and `CORS_ORIGINS` (exactly the Vercel production address, no trailing slash). `SESSION_SECRET` is generated by Render itself.
4. Name the Blueprint, click **Deploy Blueprint**. Render builds the Docker image, runs the start script (migrating Neon), and starts the API. Check the service shows **Live**, Free, Ohio, and the address `https://timetrack-demo-api.onrender.com`, which matches the one in `vercel.json`.

Step 4: prove the link works

```
curl https://<vercel-address>/api/v1/health/ready      # {"status":"ok","database":"ok"}
```

That single response proves four things: Vercel's proxy reaches Render, Render is running, the production configuration is valid, and Render can reach Neon.

Step 5: load the data and create the administrator

Run the seed and `create-admin` commands from section 7 from your laptop. Then sign in on the live site as your private administrator (a first request can take about a minute while the free API wakes), set a real password, and deactivate `admin@example.test`.

Step 6: verify and publish

1. Add the `DEMO_URL` variable and `DEMO_EMPLOYEE_PASSWORD` secret to GitHub, run **deploy-verify** (6 of 6 passed).
2. Try the real flow by hand: submit as Alex, approve as Morgan.
3. Put the live address and the employee and manager logins in the README.

8.2 Environment variables

Variable	Where set	Value / source
`APP_ENV`	Render (from `render.yaml`)	`production`: turns on secure cookies, HSTS, disables docs, enforces config checks.

`DATABASE_URL`	Render dashboard only	The Neon direct string, `postgresql+psycopg://…?sslmode=require`.
`SESSION_SECRET`	Render (generated)	Random 256-bit value; keys the session-token digest.
`CORS_ORIGINS`	Render dashboard	The Vercel production address, exactly.
`BUSINESS_TIMEZONE`, `WEEK_START_DAY`, `LOG_LEVEL`	Render (from `render.yaml`)	`Asia/Kolkata`, `MONDAY`, `INFO`.
`VITE_API_BASE_URL`	not set	The default `/api/v1` is what makes the Vercel proxy work.
`DEMO_URL` (variable), `DEMO_EMPLOYEE_PASSWORD` (secret)	GitHub	For the smoke test only.

8.3 Failures met during the real deployment

Symptom	Cause	Fix
Vercel asks for a login on one address	That was a per-deployment URL; those are protected by default	Use the production address (it was public).
Vercel offers a "multi-service project"	It detected the Python backend	Choose "Import single project" for `frontend`.
Render form offers no region	Blueprints take the region from the file	Add `region: ohio` to `render.yaml` and push before deploying (region is permanent).
Deploy exits with status 127 "…: not found"	Render does not run `dockerCommand` through a shell	Use a script: `sh scripts/render-start.sh`.
Sign-in page shows "Request failed (404)" before the API existed	The proxy pointed at a Render address with no service yet	Expected until step 3 completed.
`zsh: no matches found: …?sslmode=require`	Placeholder text pasted as a command; `?` is a wildcard in zsh	Paste the real value into a `read` prompt, one command per paste.
Demo accounts had the connection string as password	The string was pasted at every `read` prompt	Re-run the seed with a made-up password; change the admin's password at first sign-in.

8.4 Updating and rolling back the live system

- **Update:** merge to `main`. Vercel and Render redeploy on their own (Render's Blueprint has Auto Sync on, so edits to `render.yaml` apply too, which is why that file must keep the Free plan).

- **Roll back:** Vercel → Deployments → promote an earlier deployment. Render → Deploys → roll back. If a migration changed the schema, only roll code back if the old code still runs against it.
- **Change the database password:** Neon → Roles → Reset password; copy the new string from Connect; edit `DATABASE_URL` in Render's Environment tab; the service redeploys. (Optional; recommended if the string was ever exposed.)

8.5 The self-hosted path (for a real deployment)

`compose.prod.yaml` runs four containers: PostgreSQL 16 (not published), a one-shot `migrate`, the API (non-root, read-only filesystem, no Linux capabilities) and nginx serving the built frontend with a Content-Security-Policy and proxying `/api`. Only nginx publishes a port (to 127.0.0.1); TLS is terminated in front of it by your own load balancer or a proxy such as Caddy. `docs/operations.md` covers configuration, creating the first administrator, backups and restores, retention (`scripts/maintain.py purge`), and monitoring. This path was built and smoke-tested locally but never run in a real environment.

9. Third-party services and tools in detail

Free-tier figures below were read from each provider's official pages on 25 October 2026. They are not guarantees and can change at any time. Where a page did not state something (for example whether a payment card is required), it is marked as not stated.

9.1 Neon: the managed PostgreSQL database

What it is

A "serverless Postgres" service: real PostgreSQL, where the compute (the part that runs queries) can scale to zero when idle and wake on demand, and storage is separate. It is run by Neon's company, hosted on AWS.

Why it was chosen

The app needs real PostgreSQL features (row locks, triggers, CHECK constraints, JSON). Render's own free Postgres expires after 30 days, which would have made the demo disappear; Neon's free plan has no expiry date.

How the project uses it

One project, `timetrack-demo`, one branch (`production`, the default), one database (`neondb`), one role. Region AWS US East 2 (Ohio). The API and your laptop connect with the **direct** connection string over TLS (`sslmode=require`).

Free plan limits (Neon pricing page)

Item	Free plan
Price	$0 per month
Projects	100 per account
Storage	1 GB per project (20 GB across all free projects)
Compute	100 compute-unit-hours per project per month (about 400 hours of the smallest, 0.25 CU compute); autoscaling up to 2 CU
Scale to zero	Compute suspends after 5 minutes of inactivity; always on, cannot be disabled. The first query after that wakes it and is delayed (the pages read do not give the duration).
Branches	10 per project
Restore	Instant restore within a 6-hour history window; 1 manual snapshot; no scheduled backups
When limits are exceeded	Compute is suspended until the next period, or writes that grow storage fail. The page says none of the limits delete data.
Payment card	Not stated on the pages read.

Connection types

Direct (host like `ep-…-N.us-east-2.aws.neon.tech`) and **pooled** (host contains `-pooler`, goes through PgBouncer). Neon's docs say migration tools should use a direct connection, and that protocol-

level prepared statements work through the pooler. This project uses direct everywhere, which is simplest for a single small API instance. `pool_pre_ping` is enabled in the SQLAlchemy engine, so a connection that went stale while compute was suspended is replaced instead of failing the request.

Roles and passwords

The console generates the role password. **Roles** → **role menu** → **Reset password** issues a new one (effective immediately; the old one stops working on the next connection) and the connection string must then be updated everywhere it is stored; here that is only Render's `DATABASE_URL`.

Risks and exit

Free tier can be suspended if limits are hit; no scheduled backups (take your own `pg_dump`). To leave Neon: `pg_dump`, restore into any PostgreSQL 16, change `DATABASE_URL`.

9.2 Vercel: frontend hosting and the /api proxy

What it is

A hosting and deployment platform best known for front-end projects. It builds from Git, serves static files from a global CDN and can route requests.

Why it was chosen

Free static hosting that deploys from a private GitHub repository, and **rewrites to external origins**. The rewrite is the key: the app's session cookie is `SameSite=Lax` and the API checks request origins, so a page on one domain calling an API on another would lose the cookie. With the rewrite, the browser only talks to the Vercel address and Vercel forwards `/api/*` to Render.

How the project uses it

Project `timetrack-demo-kanak` on a Hobby team; Git branch `main`; Root Directory `frontend`; preset Vite; no environment variables. Build `npm run build` (TypeScript check plus Vite build, about 20 seconds); output `dist`. Each push to `main` makes a new **production deployment**; other branches and commits get **preview** deployments at their own addresses (which do not work for sign-in because their origin is not in `CORS_ORIGINS`).

What `vercel.json` does

(1) rewrite `/api/:path*` to `https://timetrack-demo-api.onrender.com/api/:path*`; (2) rewrite every other path except `/api/` to `/index.html`, so deep links like `/admin/users` work in a single-page app; (3) response headers: a same-origin Content-Security-Policy (`default-src 'self'` with scripts and styles only from the site itself), `nosniff`, frame denial, no-referrer and a restrictive Permissions-Policy; (4) long-lived caching for hashed assets. Vercel forwards the visitor's address to the origin in `x-forwarded-for` and `x-real-ip`.

Hobby plan limits

Item	Hobby (free)
Intended use	Personal projects; **non-commercial, personal use only** (fair-use guidelines). A portfolio fits; using it to sell services does not.
Projects / domains per project	200 / 50
Deployments per day	100

Included usage per month	100 GB fast data transfer, 10 GB fast origin transfer, 1,000,000 CDN requests, 1,000,000 function invocations
Builds	2 vCPU, 8 GB memory, 32 GB disk per build
Runtime logs	1 hour retained
If limits are exceeded	Wait about 30 days before the affected feature works again (no automatic billing on Hobby)
Deployment protection	Vercel Authentication on by default for preview and per-deployment URLs; the project's production domain was public when tested
Payment card	Not stated on the page read.

Not documented in the pages I read

How long a proxied request may wait for a sleeping origin; that `Set-Cookie` passes through the rewrite. Both were therefore **tested on the live site** rather than assumed: sign-in works and the session survives.

Risks and exit

Terms are non-commercial. The frontend is static files, so it can move to any static host (the only coupling is the `/api` proxy rule).

9.3 Render: the API host

What it is

A cloud platform that runs web services, databases and jobs from Git or Docker images.

Why it was chosen

It runs the project's existing Docker image unchanged on a free plan, deploys from GitHub, and gives an HTTPS address with no certificate work.

How the project uses it

One **Blueprint** (`timetrack-demo`) created from `render.yaml` , which created one **web service** `timetrack-demo-api` : runtime Docker, Dockerfile `./backend/Dockerfile` , context `./backend` , region Ohio, plan Free, branch `main` , auto-deploy on commit, health check `/api/v1/health/ready` , start command `sh scripts/render-start.sh` . **Auto Sync** is on, so changes to `render.yaml` on `main` are applied to the Blueprint automatically.

What a deploy does

Clone the repo → build the Docker image (cached layers) → push it to Render's registry → start a container with the Blueprint's command and environment variables → wait for the health check to pass → switch traffic. Logs for each deploy and live logs are in the dashboard. The start script runs `alembic upgrade head` then `uvicorn` on the `$PORT` Render supplies (default 10000).

Free web service limits (Render docs)

Item	Free
Instance hours	750 per workspace per month (enough for one service running continuously); they do not roll over; if used up, free services are suspended until the next month

Spin-down	After 15 minutes without inbound traffic. Waking takes about a minute; browsers see a loading page. The dashboard warns of delays "of 50 seconds or more".
Resources	0.1 CPU, 512 MB RAM
Storage	Local filesystem is lost on redeploy, restart or spin-down; no persistent disk (the app keeps nothing on disk)
Not available on free	Pre-deploy command (paid; hence migrations live in the start script), scaling beyond one instance, shell access, one-off jobs, private network traffic, outbound SMTP ports 25/465/587
Bandwidth and builds	Counted against monthly included amounts (the page does not give the figures). Without a payment method, overages suspend free services or disable new builds for the rest of the month instead of billing.
Intended use	Render's own wording: suitable for testing and hobby projects; "do not use them for production applications".
Render's free Postgres	Expires 30 days after creation (14-day grace, then deleted) and has no backups. **Not used**.
Payment card	Not stated as required for a free instance.

Risks and exit

Slow first request after idle; can restart at any time; trust of forwarded headers (below). The same image runs anywhere Docker runs, so moving is a matter of setting the same environment variables.

A known security trade-off

The API trusts `X-Forwarded-For` from any sender (`--forwarded-allow-ips "*"` , as in the Docker image). Behind Render and Vercel the real visitor address arrives that way, which is what lets the login throttle see individual visitors. But Render's own address is public, so someone calling it directly could forge the header and sidestep the per-address throttle. Accepted for a demo because the private administrator has a long random password; recorded as risk R-6 in the security review.

9.4 GitHub

What it is

Git hosting, collaboration and automation (GitHub Actions).

How the project uses it

Private repository; branch-per-change history; Actions for CI, weekly dependency audits and the live smoke test; **repository secrets and variables** for the smoke test (`DEMO_EMPLOYEE_PASSWORD` , `DEMO_URL`); sign-in provider for Neon, Vercel and Render; the source both Vercel and Render deploy from (each was granted access to only this one repository).

Actions runners

Fresh Ubuntu 24.04 virtual machines. Action versions used: `checkout` , `setup-python` , `setup-node` and `upload-artifact` , all at v7 (bumped earlier to leave the deprecated Node 20 runtime). Free-plan Actions minutes and storage limits were **not** checked for this document; CI runs about 16 to 21 minutes each push because of the three-browser end-to-end job.

Secrets handling

Secrets are encrypted, masked in logs and not passed to workflows triggered from forks. Anyone with collaborator access can use them in workflows.

9.5 Docker and container images

Image	Used for
`postgres:16`	The local development database (`compose.yaml`), the CI services, the self-hosted database.
`python:3.12-slim`	Base of the API image (two stages: build the virtual environment, then copy it into a clean runtime image that runs as a non-root user).
`node:22-slim` → `nginxinc/nginx-unprivileged:1.27-alpine`	Self-hosted frontend: build the React app, then serve it from an unprivileged nginx. (Not used by the Vercel deployment.)

Docker Desktop on the laptop runs the local database; `docker exec` opens a `psql` shell in it. The images were also built and run locally to test the Render start command before it was pushed.

9.6 Libraries (installed from PyPI and npm)

Backend (Python)

Package	Role
FastAPI, Pydantic, pydantic-settings	Web framework, request/response validation, typed configuration from environment variables.
uvicorn	The server that runs the app (HTTP, proxy-header handling).
SQLAlchemy 2, psycopg 3 (binary), Alembic	Database access, the PostgreSQL driver, migrations.
argon2-cffi	Argon2id password hashing.
tzdata	Time-zone data for business dates.
pytest, httpx, ruff, mypy (development)	Tests, the HTTP test client, linting and formatting, static type checking.

Frontend (JavaScript/TypeScript)

Package	Role
React 19, React DOM, React Router 7	UI and routing.
TanStack Query 5	Server-state caching, retries, invalidation.
Vite 8, @vitejs/plugin-react, TypeScript 6	Dev server, production build, type checking.
Vitest 5, Testing Library (react, user-event, jest-dom), jsdom	Component tests.

Playwright, @axe-core/playwright	Browser end-to-end tests and automated accessibility checks.
ESLint (with react-hooks and jsx-a11y), Prettier	Linting (including accessibility rules) and formatting.

No UI component library and no state-management library were added (decisions D-9 and D-10), to keep the dependency surface small. `pip-audit` and `npm audit` found no known vulnerabilities in these on 25 October 2026.

9.7 Things that were considered but not used

- **Render's free PostgreSQL** (expires after 30 days), **Supabase** (considered as an alternative; not evaluated further), **Cloudflare quick tunnels** (offered as a zero-account way to share a laptop-hosted copy, never set up), **Vercel's Python/"services" hosting** (offered in the import screen, declined), email providers (the notification outbox has no real adapter).

10. Running it afterwards: operations, costs, limits, risks

Day-to-day

- **Is it up?** `curl https://timetrack-demo-kanak.vercel.app/api/v1/health/ready` should return `{"status":"ok","database":"ok"}`. If it hangs for a minute the API was asleep.
- **Logs:** Render dashboard → the service → Logs (one JSON object per line with a request id; the same id appears in error responses). Vercel keeps one hour of runtime logs.
- **Health after a deploy:** run `deploy-verify`.

Costs and billing risk

- Everything is on free plans. None of the pages read states that a card is required; each provider's sign-up flow decides that, and any payment prompt should be read before accepting.
- Render: with no payment method, overage means suspension rather than a charge. Vercel Hobby: limits pause features; no automatic billing. Neon free: limits suspend compute or fail writes. The practical risk is the demo going offline, not a bill, *provided you stay on the free plans*. The purple banner on Render's Blueprint page ("future updates to your Blueprint file … may change your costs") means a change to `plan:` in `render.yaml` could create a paid instance, so review that file before merging changes to it.

Known limitations and risks

Item	Detail
Cold start	About a minute after 15 idle minutes; Neon adds a short wake-up on top.
Provider terms	Render: free is for hobby/testing; Vercel Hobby: non-commercial.
Shared demo password	Anyone with it can add and submit entries on the live site; reset by re-seeding.
Login throttle	In-memory and single-instance; forgeable address behind the public Render URL (risk R-6).
No scheduled backups	Take your own `pg_dump`.
No email	Notifications go to a log only.
No MFA/SSO	Local accounts only.
Not tested	Screen readers; real phones; production-scale load; the self-hosted Docker path in a real environment; restoring a backup onto another PostgreSQL major version.
Browser quirk	WebKit logs harmless CSP console messages for native <select> controls (reproduced on a bare page; no visible effect found).

Performance (measured locally, not on the live site)

On a laptop with ~15,000 entries: dashboard read about 6 ms at the 95th percentile, saving an entry about 11 ms, admin lists 4 to 8 ms, creating a user about 64 ms (Argon2 hashing), reports over a year about 42 to 65 ms. These measure the application, not the free hosts: expect extra latency from the network hops and the cold starts described above. Load and security scanners were deliberately **not** run against the public site.

Security summary

Argon2id hashes; HMAC-digested sessions in `HttpOnly` `Secure` cookies; CSRF header and origin allow-list; role guards plus a route-by-role matrix test; strict CSP, HSTS and other headers; append-only audit tables; no secrets or personal data in audit rows; CSV formula neutralising; weekly dependency audit; production-mode refusal of weak configuration. This was an engineering review by the authors, not an independent penetration test (`docs/security-review.md`).

Appendix

A. Commands cheat-sheet

Task	Command
Start / stop the local app	`scripts/dev-up.sh` / `scripts/dev-down.sh` (app on localhost:5173, API on 8000)
Backend checks	`cd backend && .venv/bin/ruff check . && .venv/bin/mypy app scripts && .venv/bin/pytest`
Frontend checks	`cd frontend && npm run lint && npm run typecheck && npm test && npm run build`
End-to-end tests	`cd frontend && npx playwright test` (add `--project=chromium` for one browser)
Performance check	`cd backend && .venv/bin/python scripts/perf_check.py`
Backup drill	`scripts/backup-restore-drill.sh`
Live health	`curl https://timetrack-demo-kanak.vercel.app/api/v1/health/ready`
Run the live smoke test	`gh workflow run deploy-verify`
Purge old rows (self-hosted)	`python scripts/maintain.py purge --dry-run`
Create an administrator	`python scripts/maintain.py create-admin EMAIL "Name"` (password from `ADMIN_PASSWORD`)

B. Where things are in the repository

Path	What
`backend/app/`	API (`api/`), business rules (`services/`), queries (`repositories/`), tables (`models/`), security helpers, tests (`tests/`)
`backend/alembic/versions/`	The five migrations
`backend/scripts/`	`seed.py`, `maintain.py`, `render-start.sh`, `perf_check.py`, `prepare_db.py`, `e2e_state.py`
`frontend/src/`	The React app: `features/` (dashboard, time-entries, approvals, admin, reports, auth), `components/`, `api/`
`frontend/tests/e2e/`, `tests/smoke/`	Browser tests; live smoke tests
`.github/workflows/`	`ci.yml`, `audit.yml`, `deploy-verify.yml`

`docs/`	architecture, decisions, data model, API contracts, testing strategy, security review, operations, performance, user guide, free-tier deployment, implementation report
`render.yaml`, `frontend/vercel.json`, `compose*.yaml`, Dockerfiles	Deployment configuration

C. Glossary

API
The server's set of web addresses (under `/api/v1`) that the browser app calls to read and change data.

Argon2id
A deliberately slow, memory-hungry password-hashing algorithm.

Blueprint (Render)
A `render.yaml` file describing the services Render should create and keep in sync.

CI
Continuous integration: automatic checks on every push.

CORS / CSRF
Browser rules about which websites may call an API / protection against a malicious site making your browser perform an action.

CSP
Content-Security-Policy: a response header limiting what a page may load or run.

Idempotency key
A client-supplied token that makes retrying a request harmless.

Migration
A versioned script that changes the database schema; applied in order and recorded.

Origin
Scheme + host + port of a web page (e.g. `https://timetrack-demo-kanak.vercel.app`).

Pooled connection
A database connection through a pooler (PgBouncer) that shares connections between many clients; the alternative is a *direct* connection.

Rewrite (Vercel)
Serving the content of a different address (here, the Render API) under your own address without the browser noticing.

Seed
A script that fills a database with starting data.

Smoke test
A short check that the main paths of a deployed system work.

Spin-down / cold start
A free service being stopped after inactivity, and the delay while it starts again.

TLS / HTTPS

Encryption of traffic between two parties; the padlock in a browser.

D. Further reading in the repository

`README.md` (quick start and live demo), `docs/user-guide.md` (every scenario with test ideas for UI, API, database, performance and security), `docs/deployment-free-tier.md` (this deployment in checklist form), `docs/operations.md` (self-hosting), `docs/security-review.md`, `docs/decisions.md`, `docs/implementation-report.md`.